

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

Департамент цифровых, робототехнических систем и электроники

**«Введение в pandas: изучение структуры Series и базовых операций»
Отчет по лабораторной работе № 4
по дисциплине «Искусственный интеллект и машинное обучение»
Вариант 5**

Выполнил студент группы ИВТ-б-о-24-1
Грабарь Артемий Павлович
«__» мая 2026г.

Подпись студента _____
Работа защищена « » _____ 20__ г.
Проверил Воронкин Р.А. _____
(подпись)

Ставрополь 2026

Цель работы: Познакомиться с основами работы с библиотекой pandas, в частности, со структурой данных Series.

Ссылка на GitHub: https://github.com/Arhi258/Laba_ML_4.git

Задание:

1. Создать общедоступный репозиторий на GitHub, в котором будет использована лицензия MIT и выбранный язык программирования.
2. Выполнить клонирование созданного репозитория на рабочий компьютер.
3. Дополнить файл .gitignore необходимыми правилами для выбранного языка программирования, интерактивной оболочки Jupyter notebook и интегрированной среды разработки.
4. Проработать примеры практической работы.
5. Решить практические задания работы.
6. Зафиксировать сделанные изменения в репозитории.
7. Отправить сделанные изменения на сервер GitHub.

Ход работы:

Был создан общедоступный репозиторий на GitHub, в котором будет использована лицензия MIT.

Было выполнено клонирование данного репозитория на рабочий компьютер.

Файл .gitignore был дополнен необходимыми правилами для работы с Anaconda. Был изменен файл README.

Было выполнено задание 1. Результат представлен на рисунке 1.

1. Создание Series из списка

Создайте `Series` из списка чисел `[5, 15, 25, 35, 45]` с индексами `['a', 'b', 'c', 'd', 'e']`. Выведите его на экран и определите его тип данных.

```
[1]: import pandas as pd

data = [5, 15, 25, 35, 45]
indices = ['a', 'b', 'c', 'd', 'e']

my_series = pd.Series(data=data, index=indices)

print("Созданный Series:")
print(my_series)
print("-" * 30)

print(f"Тип данных элементов внутри Series: {my_series.dtype}")
print(f"Тип самого объекта: {type(my_series)}")
```

```
Созданный Series:
a      5
b     15
c     25
d     35
e     45
dtype: int64
-----
Тип данных элементов внутри Series: int64
Тип самого объекта: <class 'pandas.core.series.Series'>
```

Рисунок 1. Результат выполнения задания

Было выполнено задание 2. Результат представлен на рисунке 2.

2. Получение элемента Series

Дан `Series` с индексами `['A', 'B', 'C', 'D', 'E']` и значениями `[12, 24, 36, 48, 60]`. Используйте `.loc[]` для получения элемента с индексом `'C'` и `.iloc[]` для получения третьего элемента.

```
[2]: import pandas as pd

indices = ['A', 'B', 'C', 'D', 'E']
values = [12, 24, 36, 48, 60]
my_series = pd.Series(data=values, index=indices)

element_loc = my_series.loc['C']
print(f"Элемент с индексом 'C' (.loc): {element_loc}")

element_iloc = my_series.iloc[2]
print(f"Третий элемент (.iloc): {element_iloc}")
```

```
Элемент с индексом 'C' (.loc): 36
Третий элемент (.iloc): 36
```

Рисунок 2. Результат выполнения задания

Было выполнено задание 3. Результат представлен на рисунке 3.

▼ 3. Фильтрация данных с помощью логической индексации

Создайте `Series` из массива NumPy `np.array([4, 9, 16, 25, 36, 49, 64])`. Выберите только те элементы, которые больше 20, и выведите результат.

```
[4]: import numpy as np
import pandas as pd

np_array = np.array([4, 9, 16, 25, 36, 49, 64])
my_series = pd.Series(np_array)

filtered_series = my_series[my_series > 20]

print("Элементы Series, которые больше 20:")
print(filtered_series)
```

```
Элементы Series, которые больше 20:
3      25
4      36
5      49
6      64
dtype: int64
```

Рисунок 3. Результат выполнения задания

Было выполнено задание 4. Результат представлен на рисунке 4.

4. Просмотр первых и последних элементов

Создайте `Series`, содержащий 50 случайных целых чисел от 1 до 100 (используйте `np.random.randint`). Выведите первые 7 и последние 5 элементов с помощью `.head()` и `.tail()`.

```
[5]: import numpy as np
import pandas as pd

np.random.seed(42)

random_numbers = np.random.randint(1, 101, size=50)
my_series = pd.Series(random_numbers)

print(my_series.head(7))
print(my_series.tail(5))
```

```
0      52
1      93
2      15
3      72
4      61
5      21
6      83
dtype: int32
45      51
46       7
47      21
48      73
49      39
dtype: int32
```

Рисунок 4. Результат выполнения задания

Было выполнено задание 5. Результат представлен на рисунке 5.

5. Определение типа данных Series

Создайте `Series` из списка `['cat', 'dog', 'rabbit', 'parrot', 'fish']`. Определите тип данных с помощью `.dtype`, затем преобразуйте его в `category` с помощью `.astype()`.

```
[6]: import pandas as pd

animals = ['cat', 'dog', 'rabbit', 'parrot', 'fish']
my_series = pd.Series(animals)

print(my_series.dtype)

my_series = my_series.astype('category')
print(my_series.dtype)

object
category
```

Рисунок 5. Результат выполнения задания

Было выполнено задание 6. Результат представлен на рисунке 6.

6. Проверка пропущенных значений

Создайте `Series` с данными `[1.2, np.nan, 3.4, np.nan, 5.6, 6.8]`. Напишите код, который проверяет, есть ли в `Series` пропущенные значения (`NaN`), и выведите индексы таких элементов.

```
[7]: import numpy as np
import pandas as pd

data = [1.2, np.nan, 3.4, np.nan, 5.6, 6.8]
my_series = pd.Series(data)

print(my_series.isna().any())

nan_indices = my_series[my_series.isna()].index
print(nan_indices)

True
Index([1, 3], dtype='int64')
```

Рисунок 6. Результат выполнения задания

Было выполнено задание 7. Результат представлен на рисунке 7.

7. Заполнение пропущенных значений

Используйте `Series` из предыдущего задания и замените все `NaN` на среднее значение всех непустых элементов. Выведите результат.

```
[8]: import numpy as np
import pandas as pd

data = [1.2, np.nan, 3.4, np.nan, 5.6, 6.8]
my_series = pd.Series(data)

mean_value = my_series.mean()
filled_series = my_series.fillna(mean_value)

print(filled_series)
```

```
0    1.20
1    4.25
2    3.40
3    4.25
4    5.60
5    6.80
dtype: float64
```

Рисунок 7. Результат выполнения задания

Было выполнено задание 8. Результат представлен на рисунке 8.

8. Арифметические операции с Series

Создайте два `Series` :

- `s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])`
- `s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])`

Выполните сложение `s1 + s2` . Объясните, почему в результате появляются `NaN` , и замените их на `0` .

```
[9]: import pandas as pd

s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])

result = s1 + s2
print(result)

result_filled = result.fillna(0)
print(result_filled)

a      NaN
b    25.0
c    45.0
d    65.0
e      NaN
dtype: float64
a      0.0
b    25.0
c    45.0
d    65.0
e      0.0
dtype: float64
```

Рисунок 8. Результат выполнения задания

Было выполнено задание 9. Результат представлен на рисунке 9.

9. Применение функции к Series

Создайте `Series` из чисел `[2, 4, 6, 8, 10]` . Напишите код, который применяет к каждому элементу функцию вычисления квадратного корня с помощью `.apply(np.sqrt)` .

```
[10]: import numpy as np
import pandas as pd

data = [2, 4, 6, 8, 10]
my_series = pd.Series(data)

result = my_series.apply(np.sqrt)
print(result)

0    1.414214
1    2.000000
2    2.449490
3    2.828427
4    3.162278
dtype: float64
```

Рисунок 9. Результат выполнения задания

Было выполнено задание 10. Результат представлен на рисунке 10.

▼ 10. Основные статистические методы

Создайте `Series` из 20 случайных чисел от 50 до 150 (используйте `np.random.randint`). Найдите сумму, среднее, минимальное и максимальное значение. Выведите также стандартное отклонение.

```
[11]: import numpy as np
import pandas as pd

np.random.seed(42)

random_numbers = np.random.randint(50, 151, size=20)
my_series = pd.Series(random_numbers)

print(my_series.sum())
print(my_series.mean())
print(my_series.min())
print(my_series.max())
print(my_series.std())

2062
103.1
51
149
32.28964311712935
```

Рисунок 10. Результат выполнения задания

Было выполнено задание 11. Результат представлен на рисунке 11.

▼ 11. Работа с временными рядами

Создайте `Series`, где индексами будут даты с 1 по 10 марта 2024 года (`pd.date_range(start='2024-03-01', periods=10, freq='D')`), а значениями — случайные числа от 10 до 100. Выберите данные за 5–8 марта.

```
[12]: import numpy as np
import pandas as pd

np.random.seed(42)

dates = pd.date_range(start='2024-03-01', periods=10, freq='D')
values = np.random.randint(10, 101, size=10)

time_series = pd.Series(values, index=dates)
print(time_series)

filtered_data = time_series['2024-03-05':'2024-03-08']
print(filtered_data)
```

2024-03-01	61
2024-03-02	24
2024-03-03	81
2024-03-04	70
2024-03-05	30
2024-03-06	92
2024-03-07	96
2024-03-08	84
2024-03-09	84
2024-03-10	97

Freq: D, dtype: int32

2024-03-05	30
2024-03-06	92
2024-03-07	96
2024-03-08	84

Freq: D, dtype: int32

Рисунок 11. Результат выполнения задания

Было выполнено задание 12. Результат представлен на рисунке 12.

▼ 12. Проверка уникальности индексов

Создайте `Series` с индексами `['A', 'B', 'A', 'C', 'D', 'B']` и значениями `[10, 20, 30, 40, 50, 60]`. Проверьте, являются ли индексы уникальными. Если нет, сгруппируйте повторяющиеся индексы и сложите их значения.

```
[13]: import pandas as pd

indices = ['A', 'B', 'A', 'C', 'D', 'B']
values = [10, 20, 30, 40, 50, 60]

my_series = pd.Series(values, index=indices)

print(my_series.index.is_unique)

if not my_series.index.is_unique:
    result = my_series.groupby(level=0).sum()
    print(result)

False
A      40
B      80
C      40
D      50
dtype: int64
```

Рисунок 12. Результат выполнения задания

Было выполнено задание 13. Результат представлен на рисунке 13.

▼ 13. Преобразование строковых дат в DatetimeIndex

Создайте `Series`, где индексами будут строки `['2024-03-10', '2024-03-11', '2024-03-12']`, а значениями — `[100, 200, 300]`. Преобразуйте индексы в `DatetimeIndex` и выведите тип данных индекса.

```
[14]: import pandas as pd

dates = ['2024-03-10', '2024-03-11', '2024-03-12']
values = [100, 200, 300]

my_series = pd.Series(values, index=dates)

my_series.index = pd.to_datetime(my_series.index)

print(type(my_series.index))
print(my_series)

<class 'pandas.core.indexes.datetimes.DatetimeIndex'>
2024-03-10    100
2024-03-11    200
2024-03-12    300
dtype: int64
```

Рисунок 13. Результат выполнения задания

Было выполнено задание 14. Результат представлен на рисунке 14.

14. Чтение данных из CSV-файла

Создайте CSV-файл `data.csv` со следующими данными:

```
Дата,Цена
2024-03-01,100
2024-03-02,110
2024-03-03,105
2024-03-04,120
2024-03-05,115
```

Прочитайте файл и создайте `Series`, используя "Дата" в качестве индекса.

```
[15]: import pandas as pd

csv_data = """Дата,Цена
2024-03-01,100
2024-03-02,110
2024-03-03,105
2024-03-04,120
2024-03-05,115"""

with open('data.csv', 'w', encoding='utf-8') as f:
    f.write(csv_data)

df = pd.read_csv('data.csv', index_col='Дата', encoding='utf-8')
my_series = df['Цена']

print(my_series)
```

```
Дата
2024-03-01    100
2024-03-02    110
2024-03-03    105
2024-03-04    120
2024-03-05    115
Name: Цена, dtype: int64
```

Рисунок 14. Результат выполнения задания

Было выполнено задание 15. Результат представлен на рисунке 15.

▼ 15. Построение графика на основе Series

Создайте `Series`, где индексами будут даты с 1 по 30 марта 2024 года, а значениями – случайные числа от 50 до 150. Постройте график значений с помощью `matplotlib`. Добавьте заголовок, подписи осей и сетку.

```
[16]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

np.random.seed(42)

dates = pd.date_range(start="2024-03-01", end="2024-03-30", freq="D")
values = np.random.randint(50, 151, size=len(dates))
my_series = pd.Series(values, index=dates)

plt.figure(figsize=(10, 5))
plt.plot(my_series.index, my_series.values, marker="o")
plt.title("График случайных значений за март 2024")
plt.xlabel("Дата")
plt.ylabel("Значение")
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Рисунок 15. Результат выполнения задания

Было выполнено индивидуальное задание. Результат представлен на рисунке 16.

▼ Индивидуальное задание. Преобразование и анализ погодных данных

Создайте `Series`, где индексами будут даты с 10 по 20 марта 2024 года, а значениями – случайные температуры от -5 до +10 градусов. Выделите данные за 15-17 марта, преобразуйте индекс в `DatetimeIndex`, а затем постройте график температуры.

```
[18]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

np.random.seed(42)

dates = pd.date_range(start="2024-03-10", end="2024-03-20", freq="D")
temperatures = np.random.randint(-5, 11, size=len(dates))
weather_series = pd.Series(temperatures, index=dates)

weather_series.index = pd.to_datetime(weather_series.index)

filtered_weather = weather_series["2024-03-15":"2024-03-17"]

plt.figure(figsize=(8, 4))
plt.plot(filtered_weather.index, filtered_weather.values, marker="o", color="orange")
plt.title("Изменение температуры с 15 по 17 марта 2024")
plt.xlabel("Дата")
plt.ylabel("Температура (°C)")
plt.grid(True)
plt.xticks(filtered_weather.index, filtered_weather.index.strftime("%Y-%m-%d"))
plt.tight_layout()
plt.show()
```

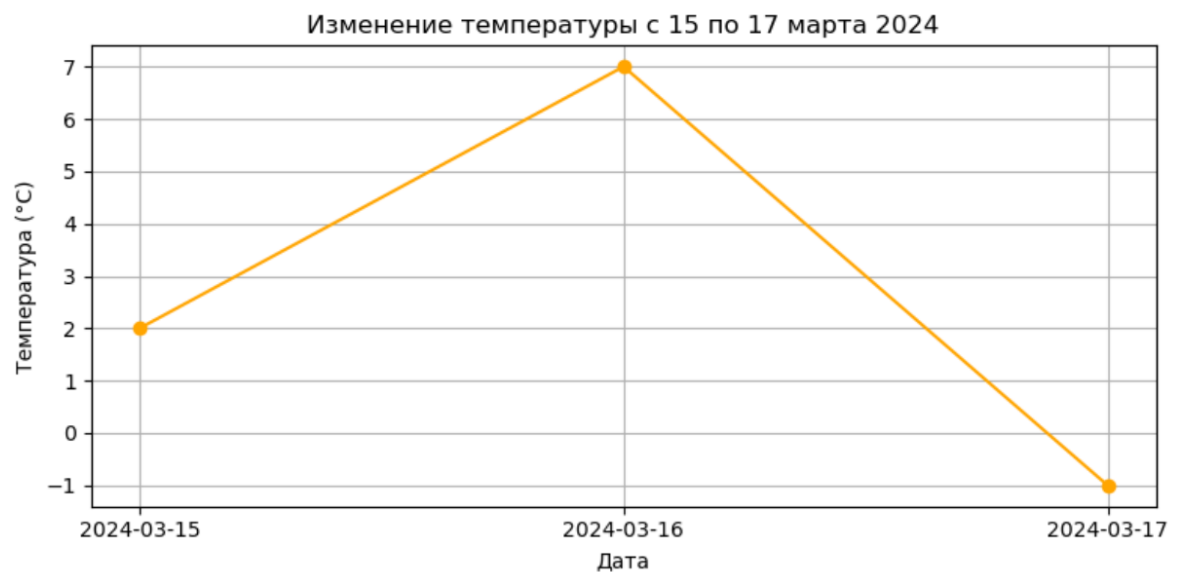


Рисунок 16. Результат выполнения задания

Все изменения были зафиксированы и отправлены на сервер GitHub.

Контрольные вопросы:

1. Что такое `pandas.Series` и чем она отличается от списка в Python? `Series` – это одномерная структура данных с индексами. В отличие от списка Python, поддерживает именованные индексы и быстрые векторные операции.

2. Какие типы данных можно использовать для создания Series?
Списки, словари, NumPy-массивы, скаляры.
3. Как задать индексы при создании Series? Через параметр `index=`.
4. Каким образом можно обратиться к элементу Series по его индексу?
Например: `s['a']` или `s[0]`.
5. В чём разница между `iloc[]` и `.loc[]` при индексации Series? `.iloc[]` – доступ по позиции, `.loc[]` — по названию индекса.
6. Как использовать логическую индексацию в Series? Через условия: `s[s > 10]`.
7. Какие методы можно использовать для просмотра первых и последних элементов Series? `.head()` и `.tail()`.
8. Как проверить тип данных элементов Series? Через `s.dtype`.
9. Каким способом можно изменить тип данных Series? Методом `.astype()`.
10. Как проверить наличие пропущенных значений в Series? С помощью `.isna()` или `.isnull()`.
11. Какие методы используются для заполнения пропущенных значений в Series? `.fillna()`, `.ffill()`, `.bfill()`.
12. Чем отличается метод `.fillna()` от `.dropna()`? `.fillna()` заполняет пропуски, `.dropna()` удаляет строки с пропусками.
13. Какие математические операции можно выполнять с Series?
Сложение, вычитание, умножение, деление и другие.
14. В чём преимущество векторизованных операций по сравнению с циклами Python? Они работают быстрее и требуют меньше кода.
15. Как применить пользовательскую функцию к каждому элементу Series? Через `.apply(func)`.
16. Какие агрегирующие функции доступны в Series? `.sum()`, `.mean()`, `.min()`, `.max()`, `.count()` и др.
17. Как узнать минимальное, максимальное, среднее и стандартное отклонение Series? Методами `.min()`, `.max()`, `.mean()`, `.std()`.

18. Как сортировать Series по значениям и по индексам? `.sort_values()` — по значениям, `.sort_index()` — по индексам.

19. Как проверить, являются ли индексы Series уникальными? Через `s.index.is_unique`.

20. Как сбросить индексы Series и сделать их числовыми? `.reset_index(drop=True)`.

21. Как можно задать новый индекс в Series? `s.index = [...]` или `.set_axis()`.

22. Как работать с временными рядами в Series? Использовать `DatetimeIndex` и методы для дат и времени.

23. Как преобразовать строковые даты в формат `DatetimeIndex`? С помощью `pd.to_datetime()`.

24. Каким образом можно выбрать данные за определённый временной диапазон? Например: `s['2024-01':'2024-03']`.

25. Как загрузить данные из CSV-файла в Series? Через `pd.read_csv()` и выбор нужного столбца.

26. Как установить один из столбцов CSV-файла в качестве индекса Series? Использовать параметр `index_col=`.

27. Для чего используется метод `.rolling().mean()` в Series? Для вычисления скользящего среднего.

28. Как работает метод `.pct_change()`? Какие задачи он решает? Вычисляет процентное изменение между соседними значениями; используется для анализа роста и доходности.

29. В каких ситуациях полезно использовать `.rolling()` и `.pct_change()`? При анализе временных рядов, трендов и изменений данных.

30. Почему NaN могут появляться в Series, и как с ними работать? Из-за отсутствующих данных или ошибок вычислений; их можно удалить (`.dropna()`) или заполнить (`.fillna()`).

Вывод: В результате работы были ознакомлены с основами работы с библиотекой `pandas`, в частности, со структурой данных Series.

